

AURA Forge Ops-Ready Runbook

Current evidence packet · 3 tracked Markdown source(s) · 2026-09-01

Monitoring and Incident Runbook

Source: docs/capstone/monitoring-incident-runbook.md

This runbook applies to the isolated AURA Forge capstone workflow. It is not a production FitGPT deployment runbook.

Monitored Signals

Signal	Source	Action threshold
Policy gate failure	GitHub Actions policy-tests	Stop. Do not merge until failure is diagnosed and fixed.
Evaluation gate failure	GitHub Actions evaluation-gate	Stop. Determine whether the failure is workflow config, dependency, or real regression.
Pipeline integrity failure	GitHub Actions pipeline-integrity	Stop. Treat as guardrail weakening until proven otherwise.
Advisory unavailable	GitHub Actions advisory-review	Continue if permanent gates pass and artifact says SKIPPED - AI SECRET UNAVAILABLE.
Audit trail failure	GitHub Actions audit-trail	Stop for evidence repair unless upstream permanent failure already explains unavailable artifacts.
Role overreach denial	MCP authorization result	Expected for unauthorized attempts; preserve denial as safety evidence.
Holdout checksum mismatch	shasum -a 256 docs/holdout-task-set.md	Stop. Restore integrity without rewriting history.
Secret-pattern finding	local credential scan or GitHub secret scanning	Stop. Do not push further. Rotate if a real secret was exposed.

Incident Steps

- Classify the incident as policy/eval/integrity, advisory-only, audit/evidence, environment/dependency, or security.
- Preserve the failed run ID, commit SHA, failing job, and sanitized log excerpt.
- Reproduce locally where possible without production access.
- Apply the smallest evidence-backed fix.
- Run local deterministic verification.
- Commit normally on capstone/aura-forge.
- Push normally to origin/capstone/aura-forge.
- Confirm the next GitHub run passes permanent gates.

Failure Response Matrix

Failure type	Signal	Severity	Immediate action	Owner/role	Rollback/escalation step	Evidence to capture	Recovery verification
Classifier failure	eval/test_risk_classifier.py failure or unexpected risk_tier in CI classification artifact	High	Stop routing. Treat unknown or malformed evidence as HIGH until fixed.	Tester, then Reviewer	Revert classifier rule change or route to HIGH with ESCALATION REQUIRED.	Paths, metadata, triggered rules, failing test output.	pytest -q -p no:cacheprovider eval/test_risk_classifier.py eval/test_ci_change_classifier.py
Router malformed output	eval/test_adaptive_router.py failure, missing route ID, missing gates, or unknown tier	High	Stop. Do not allow role execution from malformed plan.	Tester	Revert router change or preserve failed evidence and require plan approval.	Classifier result, route dict, failing assertion.	pytest -q -p no:cacheprovider eval/test_adaptive_router.py
Policy gate failure	GitHub policy-tests failure or local policy/MCP failure	Blocking	Stop. Do not merge or mark ready.	Tester, Reviewer	Revert permission widening or repair allow-list/test mismatch.	Run ID, job log, changed grant files, policy report.	Governed Docker eval/test_policy.py eval/test_mcp_runtime.py eval/test_coursetools_runtime.py
Evaluation failure	GitHub evaluation-gate failure or local eval regression	Blocking	Stop. Separate environment failure from real regression.	Tester	Revert the change if regression is real; repair workflow if environment/config failure.	Run ID, failing test names, sanitized log excerpt.	Relevant focused pytest plus CI green evaluation gate.

Failure type	Signal	Severity	Immediate action	Owner/role	Rollback/escalation step	Evidence to capture	Recovery verification
MCP authorization denial	Denial from governed MCP layer	Expected when unauthorized; high if unexpected for an allowed role	Preserve denial. Confirm no external state changed.	Reviewer, Project Manager for allowed-role disputes	If denial is expected, keep as safety evidence. If unexpected, escalate to HIGH and inspect allow-list.	Role, tool, decision, reason, external-state flag.	pytest -q -p no:cacheprovider eval/test_governance_overreach.py eval/test_policy.py
GitHub CI environment failure	Missing command, missing dependency, skipped required artifact	Medium to blocking depending on job	Preserve run. Repair workflow prerequisites without changing measured results.	Tester	Normal fix commit; do not force-push.	Run ID, job name, exact missing prerequisite, artifact behavior.	Next GitHub Actions run passes permanent gates.
Advisory AI unavailable	Advisory artifact says SKIPPED - AI SECRET UNAVAILABLE	Low if permanent gates pass	Continue. Do not fabricate model review.	Reviewer	None unless advisory becomes blocking or secret is exposed.	Advisory artifact and audit-trail summary.	Permanent gates pass and audit records advisory skip.
Change Passport producer missing	scripts/build-change-passport.py or eval/test_change_passport.py reports missing producer	High for submission evidence	Stop. Do not manually fill missing fields.	Tester, Project Manager	Regenerate from real producers or leave field absent with explanation.	Missing producer path, scenario ID, builder error.	pytest -q -p no:cacheprovider eval/test_change_passport.py and successful Passport generation.
Secret or forbidden file finding	Secret scan or forbidden-file scan flags submission-facing artifact	Blocking	Stop. Do not push. Remove or sanitize; rotate if a real secret was exposed.	Project Manager	If already pushed, follow credential rotation/history remediation outside this capstone run.	File path only; do not print secret value.	Secret scan returns zero findings.

Rollback Path

If a fix worsens the system, create a normal revert commit on capstone/aura-forge and rerun CI. Do not force-push or rewrite evidence history.

Tested Examples

- Run 31512923419 failed because evaluation-gate lacked pytest and audit handling assumed an unavailable integrity artifact. Commit 92d60c4 repaired both issues and the next run passed.
- Controlled tool-evolution commit 7c666e5 intentionally weakened the permanent policy gate. The integrity checker rejected the change. Commit 4e446f6 restored the contract.
- Local pipeline-integrity demo PI-20260811-001 added continue-on-error: true to policy-tests in a detached worktree. The checker returned FAIL.
- Local escalation regression eval/test_adaptive_router.py::test_low_route_observing_high_sensitive_path_fails_closed_with_escalation proves a LOW route observing mcp/coursetools_server.py returns ESCALATION REQUIRED evidence instead of silently continuing.

Graceful Degradation

Advisory AI review is optional. If AURA_ADVISORY_AI_KEY is unavailable, the workflow produces an explicit sanitized skip artifact and permanent gates continue.

No production service, production database, production secret, or live user is required for any deterministic gate.

AURA Forge Ops-Ready Runbook

Current evidence packet · 3 tracked Markdown source(s) · 2026-09-01

Timeout, Retry, and Budget Enforcement

Source: docs/capstone/reliability-controls.md

Purpose

The grader reported that timeout, retry, and budget controls were described but not demonstrated. eval/reliability_controls.py now makes each decision deterministic and testable from recorded execution evidence.

The control is separate from the frozen aura-router-v1 measurement. It does not rewrite historical scenario results.

Public Decision Contract

evaluate_execution_limits(limits, evidence) accepts:

- a positive timeout;
- an attempt limit of at least one;
- a positive model-cost limit;
- an explicit retryable-failure allow-list;
- the current attempt number, elapsed seconds, recorded cost, and outcome.

It returns one fail-closed decision:

Decision	Meaning
COMPLETED	The attempt succeeded within every limit.
RETRY_ALLOWED	The failure is allow-listed and another bounded attempt remains.
RETRY_LIMIT_REACHED	The configured attempt count is exhausted.
TIMEOUT_EXCEEDED	Elapsed time reached or exceeded the limit.
BUDGET_EXCEEDED	Recorded cost reached or exceeded the limit.
ESCALATION_REQUIRED	The failure is not explicitly retryable.

Timeout and budget checks take precedence over retry permission. A retryable provider failure cannot continue once either boundary is reached.

Demonstrated Cases

eval/test_reliability_controls.py proves:

- one retryable failure can receive one bounded retry;
- a third attempt is denied when max_attempts is two;
- timeout stops a retryable failure;
- budget exhaustion stops a retryable failure;
- a policy denial escalates without retry.

Run:

```
pytest -q -p no:cacheprovider eval/test_reliability_controls.py
```

The permanent GitHub evaluation gate includes this test.

Limitation

This is deterministic policy enforcement over recorded attempt evidence. It does not claim that a paid model call was rerun for this revision, and it does not alter the locked historical holdouts.

AURA Forge Ops-Ready Runbook

Current evidence packet · 3 tracked Markdown source(s) · 2026-09-01

Reproducibility Runbook

Source: docs/capstone/reproducibility-runbook.md

This runbook lets a grader verify AURA Forge without paid model calls, production access, or secrets.

Fast Path - Under 5 Minutes

From repository root:

```
pytest -q -p no:cacheprovider eval/test_risk_classifier.py eval/test_adaptive_router.py eval/test_pre_aura_control.py
pytest -q -p no:cacheprovider eval/test_ci_change_classifier.py eval/test_pipeline_integrity.py eval/test_audit_trail.py eval/test_o
pytest -q -p no:cacheprovider eval/test_config_docs_consistency.py eval/test_governance_overreach.py
pytest -q -p no:cacheprovider eval/test_retrieval_behavior.py eval/test_reliability_controls.py eval/test_sandbox_contract.py
python3 scripts/check-pipeline-integrity.py .github/workflows/ci.yml
python3 scripts/build-change-passport.py AF-HIGH-001 --output /tmp/aura-passport.json
```

Expected local result from the final packaging run:

Run the commands above and use their current output as the result. The historical `82 passed` count is no longer authoritative because

Governed Runtime Path

The trusted local course image is:

```
agentic_engineer_4:latest
```

If needed, rebuild from the trusted repository Dockerfile:

```
docker build -t agentic_engineer_4:latest -f Dockerfile .
```

Run the complete governed verification with a read-only root filesystem, read-only workspace, no network, no Linux capabilities, no credential mount, bounded CPU/memory/processes, and a unique container name:

```
./scripts/run-offline-governance-verification.sh
```

Expected final local result:

Current revision result:

32 passed

The earlier `18 passed` count predates the retrieval, reliability, and sandbox-contract tests.

Host Python may not have the MCP package installed. The governed runtime is the authoritative path for MCP/policy verification.

For parallel sessions, each invocation generates a unique container name from UTC time and its process ID. A caller may provide a unique AURA_VERIFY_RUN_ID; duplicate active names fail instead of sharing a container.

The offline verifier needs no Claude authentication volume and performs no paid model call.

Scenario Demonstration Path

Show deterministic classification and routing:

```
PYTHONPATH=eval python3 - <<'PY'
from risk_classifier import classify_change
from adaptive_router import build_route_plan
scenarios = {
    "LOW": ["docs/features/accessibility.md"],
    "MEDIUM": ["web/src/utils/feedbackPrompts.test.js"],
    "HIGH": ["eval/risk_classifier.py", "mcp/coursetools_server.py"],
}
for label, paths in scenarios.items():
    classification = classify_change(paths)
    route = build_route_plan(classification)
    print(label, classification.tier, route.route_id, " -> ".join(route.model_roles))
PY
```

Expected route IDs:

- LOW -> aura-low-v1
- MEDIUM -> aura-medium-v1
- HIGH -> aura-high-v1

Governance Denial Demo

Inspect the real denial:

```
sed -n '1,180p' docs/capstone/governance-overreach-demo.md
pytest -q -p no:cacheprovider eval/test_governance_overreach.py
```

The denial ID is GO-20260811-001.

Escalation / Rollback Check

The adaptive router has a fail-closed escalation test for LOW work that observes a HIGH-sensitive path during implementation:

```
pytest -q -p no:cacheprovider eval/test_adaptive_router.py::test_low_route_observing_high_sensitive_path_fails_closed_with_escalation
```

Expected result:

```
1 passed
```

This is regression/governance coverage only. It does not change the frozen measured AURA scenario results.

GitHub CI Evidence

Terminal verified run:

<https://github.com/Muhammad7839/FitGPT-Agent-Engineering/actions/runs/31513596822>

Expected statuses:

- policy-tests: success
- evaluation-gate: success
- pipeline-integrity: success
- advisory-review: success with SKIPPED - AI SECRET UNAVAILABLE
- audit-trail: success

Safety

No production environment is required. No deterministic gate requires secrets. Advisory AI review safely skips when its optional secret is unavailable. Do not deploy to Render or Vercel for grading.